



Kubernetes GPU Integration

for MLOps Workloads

A Modern Implementation Guide for
Production-Ready GPU Computing Infrastructure

June 1, 2026

Docker + Minikube + Kubernetes + GPU

Author

William Rodriguez

Líder de IA y Arquitecto de Soluciones

eCaptureDtech

Badajoz, Extremadura, España

Contact Information:

LinkedIn: es.linkedin.com/in/wisrovi-rodriguez

Email: william.rodriguez@ecapturedtech.com

GitHub: github.com/wisrovi

Expert in MLOps and Cloud Architecture

Contents

Contents	1
List of Figures	2
List of Tables	3
Listings	4
1 Executive Summary	5
2 Introduction	7
2.1 Background	8
3 Technical Architecture	9
3.1 Core Pillars of WPipe	10
4 Installation and Editor Extensions	11
4.1 WPipe Core Installation	12
4.2 WPipe Extension for Visual Studio Code	12
4.2.1 Intelligent Code Snippets	12
4.2.2 Schema Validation and IntelliSense	12
4.2.3 Pipeline Visualization and Explorer	12
4.2.4 Customizing the Developer Environment	13
5 Advanced Orchestration Components	14
5.1 Conditional Branching (Condition)	15
5.2 Iterative Loops (For)	15
5.3 Background Tasks (Background)	15
6 Resilience and Data Management	17
6.1 Checkpointing Strategy	18
7 Performance and Use Cases	19
7.1 Benchmark Results	20
8 Conclusions and Roadmap	21
9 Bibliography	23
Bibliography	24
A Advanced Settings	25
A.1 VS Code Extension Config	26

List of Figures

List of Tables

4.1	Essential Python Snippets for WPipe	12
7.1	Component Performance Overview	20

Listings

4.1	Installing WPipe	12
5.1	Conditional Logic Example	15
5.2	Iterative Loop with Validation	15
5.3	Non-blocking Background Task	15

Chapter 1

Executive Summary

The rapid evolution of Machine Learning Operations (MLOps) requires robust, scalable, and resilient infrastructure. This documentation details the integration of Kubernetes with GPU-accelerated environments using the WPipe framework. WPipe v2.4.0 introduces sophisticated orchestration primitives including `Parallel` execution, `Condition` branching, `For` loops, and `Background` tasks. These features, combined with intelligent checkpointing and nested pipeline support, enable the construction of highly efficient MLOps workflows. Furthermore, official editor extensions for Visual Studio Code enhance developer productivity through specialized snippets and validation tools. This guide serves as the definitive reference for building and maintaining production-ready GPU computing infrastructure powered by WPipe.

Chapter 2

Introduction

2.1 Background

Contemporary MLOps requires coordination of concurrent tasks across distributed nodes. Frameworks like WPipe have emerged as powerful engines offering native parallelism and strict data contracts. By integrating WPipe with Kubernetes and GPU resources, organizations can maximize throughput for training and inference phases.

Chapter 3

Technical Architecture

3.1 Core Pillars of WPipe

WPipe v2.4.0 is engineered around several core pillars that allow for complex logic definition:

- **step:** Atomic unit with retry and timeout support.
- **Condition:** Dynamic branching logic.
- **For:** Iterative loops with validation.
- **Parallel:** Native concurrent execution.
- **Background:** Non-blocking daemon operations.

Chapter 4

Installation and Editor Extensions

4.1 WPipe Core Installation

Install via `pip`:

```
1 pip install wpipe
```

Listing 4.1: Installing WPipe

4.2 WPipe Extension for Visual Studio Code

The WPipe ecosystem includes a professional-grade extension for Visual Studio Code (located in `editors/vscode/`), designed to accelerate the development lifecycle of complex pipelines.

4.2.1 Intelligent Code Snippets

The extension provides a comprehensive set of snippets that minimize boilerplate code and prevent syntax errors. These are available in both Python and YAML files.

Table 4.1: Essential Python Snippets for WPipe

Prefix	Component	Description
<code>wp-step</code>	<code>@step</code>	Generates a decorated step function with default parameters.
<code>wp-pipe</code>	<code>Pipeline</code>	Boilerplate for a new synchronous pipeline instance.
<code>wp-parallel</code>	<code>Parallel</code>	Template for parallel step execution blocks.
<code>wp-for</code>	<code>For</code>	Loop structure with validation expression placeholders.
<code>wp-ctx</code>	<code>PipelineContext</code>	Typed class for data contract definition.

4.2.2 Schema Validation and IntelliSense

WPipe pipelines can be configured via YAML. The extension integrates a custom JSON Schema that validates configurations in real-time.

- **Type Checking:** Detects incorrect types for timeouts, retry counts, and worker limits.
- **Autocompletion:** Provides suggestions for available keys within `pipeline_config` blocks.
- **Hover Info:** Displays documentation for specific parameters directly within the editor.

4.2.3 Pipeline Visualization and Explorer

The extension adds a dedicated sidebar icon to VS Code, enabling:

- **Graph Preview:** A live view of the pipeline's DAG (Directed Acyclic Graph) as defined in code.
- **Execution Logs:** Integration with the local SQLite database to display recent execution statuses.
- **Environment Validation:** Checks if the current Python environment has `wpipe` installed and correctly configured.

4.2.4 Customizing the Developer Environment

Administrators can extend the extension's behavior by modifying `.vscode/settings.json`. This allows for defining custom step registries or adjusting the default port for the integrated dashboard.

Chapter 5

Advanced Orchestration Components

5.1 Conditional Branching (Condition)

Branching logic is handled via expression evaluation.

```
1 from wpipe import Pipeline, step, Condition
2
3 @step(name="procesar")
4 def procesar(data): return {"resultado": "ok"}
5
6 @step(name="alerta")
7 def alertar(data): return {"alerta": "error"}
8
9 pipeline = Pipeline(pipeline_name="condicional")
10 pipeline.set_steps([
11     Condition(
12         expression="valor > 100",
13         branch_true=[procesar],
14         branch_false=[alertar]
15     )
16 ])
```

Listing 5.1: Conditional Logic Example

5.2 Iterative Loops (For)

Loops with early-exit validation.

```
1 from wpipe import Pipeline, step, For
2
3 @step(name="conducir")
4 def conducir(data):
5     return {"nivel_gasolina": data.get("nivel_gasolina", 100) - 10}
6
7 viaje = Pipeline(pipeline_name="bucle")
8 viaje.set_steps([
9     For(
10         iterations=10,
11         validation_expression="nivel_gasolina > 0",
12         steps=[conducir]
13     )
14 ])
```

Listing 5.2: Iterative Loop with Validation

5.3 Background Tasks (Background)

Fire-and-forget asynchronous execution.

```
1 from wpipe import Pipeline, step
2 from wpipe.pipe.components.logic_blocks import Background
3
4 @step(name="telemetria")
5 def enviar_telemetria(data):
```

```
6     import time
7     time.sleep(2)
8     print("Telemetry sent!")
9
10    p = Pipeline(pipeline_name="background_demo")
11    p.set_steps([
12        Background(enviar_telemetria),
13        tarea_principal
14    ])
```

Listing 5.3: Non-blocking Background Task

Chapter 6

Resilience and Data Management

6.1 Checkpointing Strategy

State persistence across pod lifecycles.

Chapter 7

Performance and Use Cases

7.1 Benchmark Results

Table 7.1: Component Performance Overview

Component	Optimization Type	Typical Gain
Parallel (I/O)	ThreadPool	3x - 5x
Parallel (CPU)	ProcessPool	1.5x - 2x
Background	Daemon Thread	Immediate return

Chapter 8

Conclusions and Roadmap

WPipe 2.4.0 provides a comprehensive toolset for MLOps.

Chapter 9

Bibliography

Bibliography

[1] W. Rodriguez, *WPipe Documentation*, 2024.

Appendix A

Advanced Settings

A.1 VS Code Extension Config

The extension configuration can be found in `.vscode/settings.json`.